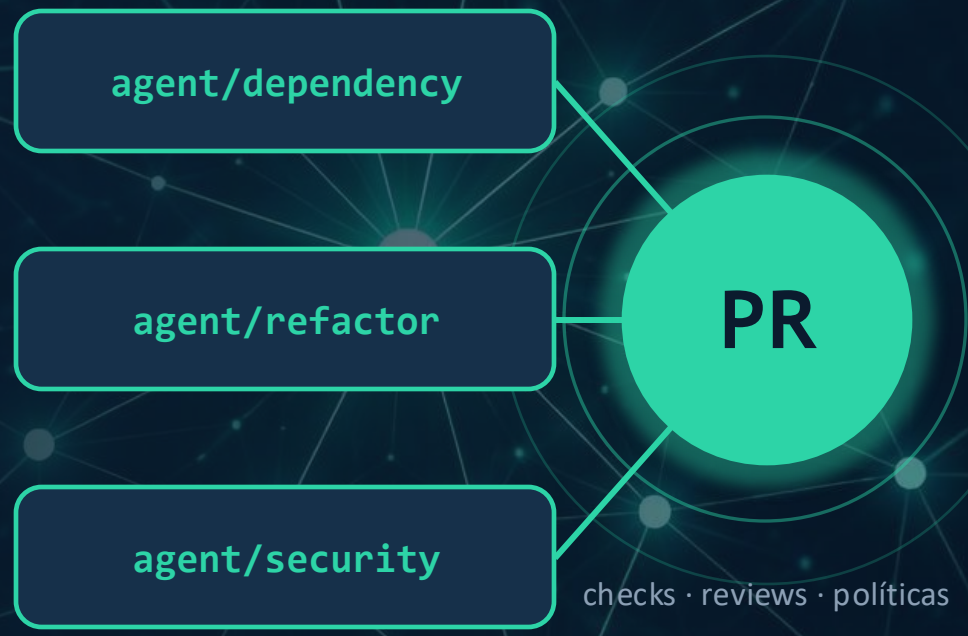
COORDINACIÓN · CONTROL · TRAZABILIDAD

GitHub
Sistemas multiagente

Cómo coordinar múltiples agentes sobre un mismo repositorio de forma segura: responsabilidades, aislamiento, evidencia y control.



Los agentes proponen. Las políticas y las personas deciden.

INTRODUCCIÓN

El desafío: coordinar varios agentes de forma segura

Agentes con responsabilidades distintas

- **Actualización de dependencias**
- **Corrección de vulnerabilidades**
- **Refactorización de código**
- **Mejora de documentación**
- **Reportes del repositorio**
- **Validación de cambios**

Sin una estrategia clara, los agentes pueden…

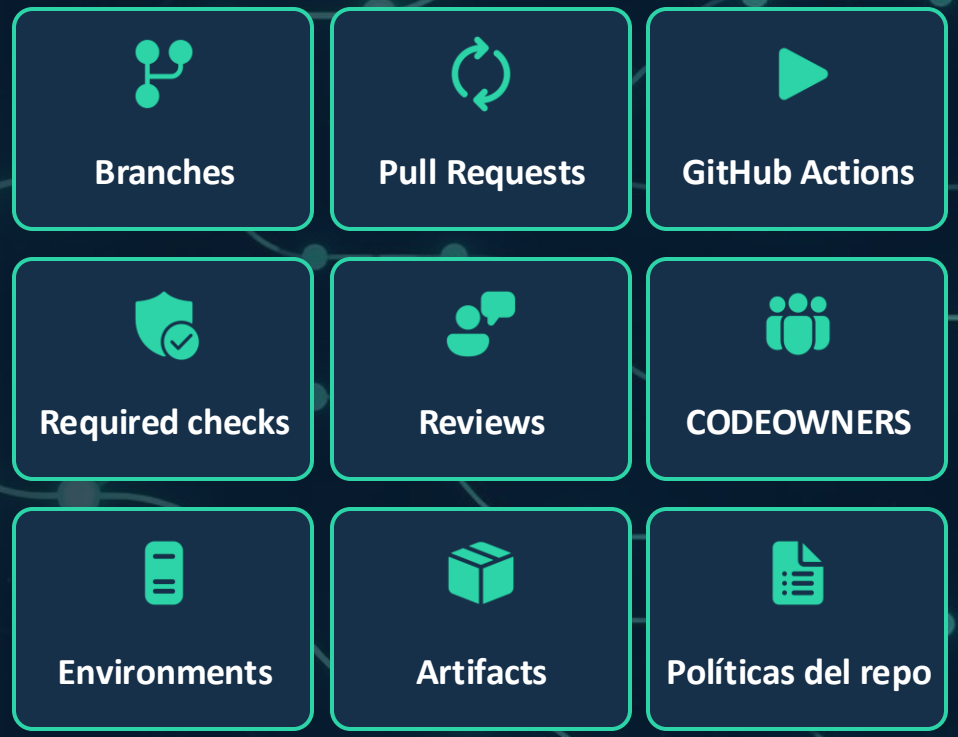
- Modificar los mismos archivos
- Crear Pull Requests que se superponen
- Duplicar trabajo
- Ejecutar workflows repetidamente
- Generar conflictos entre cambios
- Entrar en ciclos sin convergencia

Ya no importa solo si un agente puede hacer una tarea: importa cómo coordinar a todos de manera segura.

IDEA CENTRAL

Los agentes proponen cambios. Las políticas y las personas determinan si esos cambios son aceptados.

GitHub coordina mediante elementos nativos



GitHub como plataforma de coordinación, control y trazabilidad.

Cada agente, una responsabilidad y un alcance claros

Agente	Responsabilidad	Alcance: qué puede modificar
Dependency Agent	Actualizar dependencias y lockfiles	Manifiestos y lockfiles
Refactoring Agent	Modificar código de src/	src/, pero no dependencias ni workflows
Security Agent	Analizar vulnerabilidades y validar resultados	Análisis, reportes o correcciones de vulnerabilidades
Documentation Agent	Actualizar documentación	Documentación

Buena práctica: definir qué puede modificar cada agente y qué debe evitar modificar. Reduce la superposición y facilita la revisión de los cambios.

Las responsabilidades siguen el ciclo de desarrollo

Planning	Implementation	Validation	Acceptance	Deployment
01	02	03	04	05
RESPONSABILIDAD	RESPONSABILIDAD	RESPONSABILIDAD	RESPONSABILIDAD	RESPONSABILIDAD
Definir objetivo, alcance y riesgos	Realizar cambios	Generar evidencia	Revisar y aprobar	Controlar despliegues
ELEMENTO DE GITHUB	ELEMENTO DE GITHUB	ELEMENTO DE GITHUB	ELEMENTO DE GITHUB	ELEMENTO DE GITHUB
PR / PLAN.md	**Branch + commits + PR**	**Actions + checks + artifacts**	**CODEOWNERS + reviews**	**Environments + approvals**

Cada etapa del ciclo tiene un responsable claro y un elemento de GitHub que deja evidencia.

Orquestación con GitHub Actions y eventos

DISPARADORES HABITUALES

- `schedule`
- Eventos de Pull Requests
- Fin de otros workflows
- `workflow_dispatch`

Orquestación secuencial

Una tarea depende del resultado de otra.

1. Un agente actualiza una dependencia y crea un PR
2. El workflow de CI ejecuta las pruebas
3. Se ejecuta una validación de seguridad
4. Una persona revisa el cambio
5. El PR se fusiona cuando se cumplen todas las condiciones

Orquestación paralela

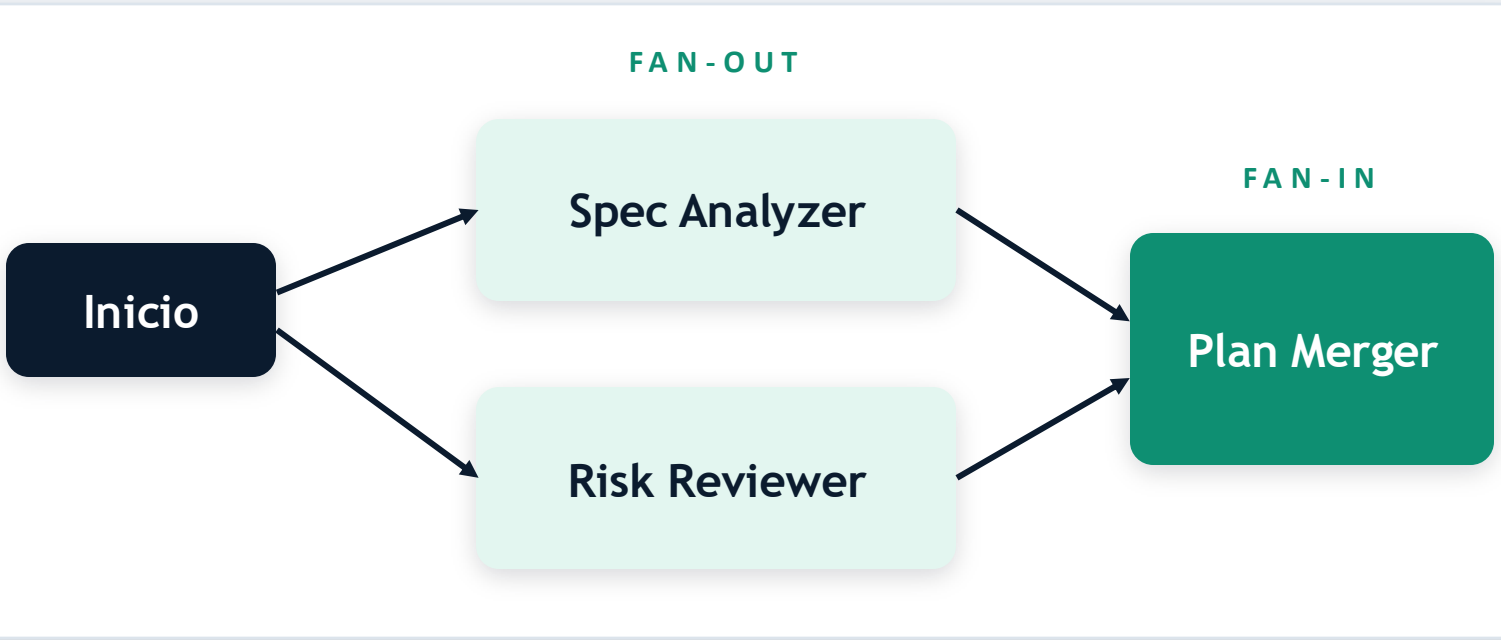
Varios agentes trabajan a la vez sin interferirse.

Documentation Agent
Actualiza documentación →

Refactoring Agent
Refactoriza código →

Siempre que sus ámbitos estén correctamente aislados.

Patrón Fan-Out / Fan-In



En GitHub Actions, la dependencia entre jobs se define mediante needs.

```
plan-merger:
  needs:
    - spec-analyzer
    - risk-reviewer
```

Fan-out

Los primeros trabajos se ejecutan en paralelo.

Fan-in

El trabajo que espera sus resultados y los combina.

3 · ARTIFACTS

Coordinación mediante artifacts compartidos

01
Ejecuta una tarea

02
Genera un resultado estructurado

03
Lo publica como artifact

04
Otro workflow o proceso lo utiliza

Razonamiento → **Evidencia** → **Ejecución controlada**

Evitar dependencias ocultas: los artifacts son revisables y conservan evidencia de las acciones de cada agente.

EJEMPLOS DE ARTIFACTS

- Reportes
- Resultados de análisis
- Planes
- Archivos JSON
- Resultados de validaciones

Aislamiento de ejecución: branches, workflows y permisos

Branches

Cada agente trabaja desde una branch independiente.

```
agent/dependency/ticket-123
agent/refactor/ticket-456
agent/security/ticket-789
```

Se identifica fácilmente qué agente realizó cada cambio.

Workflows

Workflows independientes o jobs separados con permisos específicos. Se controla cada uno:

- **Triggers**
- **Permisos**
- **Outputs**
- **Validaciones**

Control individual por agente.

Permisos

Principio de mínimo privilegio: solo los permisos necesarios.

```
permissions:
  contents: read
  pull-requests: write
```

Ejemplo: leer el repositorio y modificar información de un PR.

Control de concurrencia con concurrency

```yaml
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

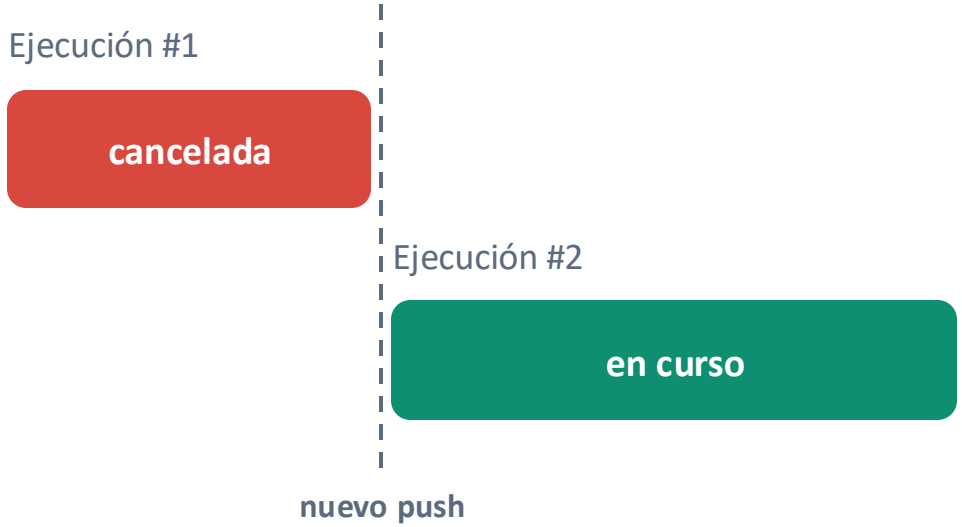
group

Aísla la concurrencia por workflow y branch.

cancel-in-progress

Si empieza una nueva ejecución del mismo workflow en la misma branch, la anterior puede cancelarse.

Dos ejecuciones sobre la misma branch



Especialmente útil cuando un Pull Request recibe actualizaciones frecuentes.

Detección y resolución de conflictos

Conflictos de merge

Dos agentes modifican los mismos archivos o líneas.

Conflictos semánticos

Se fusionan técnicamente, pero el resultado combinado rompe el comportamiento de la aplicación.

Conflictos de políticas

Algunos archivos necesitan revisiones o aprobaciones especiales.

Trabajo duplicado

Dos agentes intentan resolver el mismo problema con soluciones diferentes.

CODEOWNERS

Define qué personas o equipos revisan los cambios según las rutas modificadas.

```
/security/ @security-team
/.github/workflows/ @platform-team
/infra/ @platform-team
* @core-team
```

Los cambios sensibles llegan solos a sus responsables.

Se controlan con: Pull Requests · Required checks · Branch protection · CODEOWNERS · Reviews · GitHub Actions

Observabilidad: qué hizo cada agente, cuándo y por qué

GITHUB DEJA TRAZABILIDAD EN

- Pull Requests
- Workflow runs
- Checks
- Logs
- Artifacts
- Commits
- Issues
- Reviews

Convención en títulos de PR

```
[agent: dependency] Update package
[agent: security] Fix vulnerability
[agent: refactor] Refactor authentication module
```

Labels

```
agent: dependency
agent: security
agent: refactor
```

Una acción importante debe dejar evidencia persistente dentro del repositorio.

El Pull Request como punto de transferencia

ESTRUCTURA DEL PR	QUEDA REGISTRADO
`## Objective`	**El objetivo**
`## Plan`	**El plan**
`## Evidence`	**La evidencia**
`## Decisions and handoffs`	**Las decisiones y los responsables**
`## Risks and rollback`	**Los riesgos y la estrategia de recuperación**

Handoff: transferir una tarea desde un agente hacia otro agente o hacia una persona.

Diagnóstico de fallos en sistemas multiagente

Ejecución parcial

El agente crea un PR pero no genera la evidencia o validación esperada.

Workflow bloqueado

Los checks fallan repetidamente o nunca llega una aprobación.

Outputs incompatibles

Dos PRs intentan introducir cambios que no pueden coexistir.

Flapping

Los workflows se ejecutan repetidamente sin llegar a un estado estable.

SE INVESTIGAN CON

- Timeline del Pull Request
- Required checks
- Historial de GitHub Actions
- Logs
- Artifacts generados

El sistema puede fallar aunque cada agente, por separado, funcione correctamente.

Recuperación ante errores y escalamiento a personas

Retry → **Límite** → **Escalamiento humano**

Reintentos limitados
Hay un número máximo de intentos, para evitar ciclos infinitos.

Rollback
Cambios pequeños y aislados que se revierten fácilmente.

Escalamiento
Tras los intentos permitidos, el proceso se detiene y pide intervención humana.

Escalar cuando…

- Un conflicto continúa después de dos intentos
- Un required check falla dos veces por la misma causa
- Dos agentes proponen soluciones incompatibles
- Una operación requiere una decisión de alto riesgo

El escalamiento debe informar

- Qué ocurrió
- Qué intentó hacer el agente
- Qué resultado obtuvo
- Qué opciones existen

Control de operaciones de alto riesgo con Environments



Operaciones sensibles

Por ejemplo, los despliegues en producción se protegen con GitHub Environments.

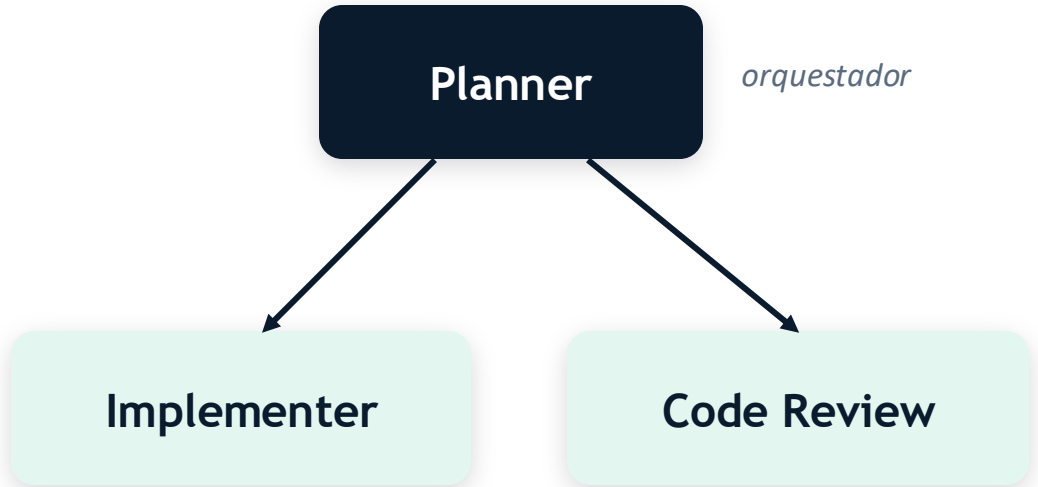
Aprobación previa

Un environment llamado production puede exigir aprobación antes de que el workflow continúe.

Barrera adicional

Se interpone entre la automatización y las operaciones críticas.

Subagentes y handoffs



Agente principal: orquestador

Delega tareas específicas en subagentes especializados.

Handoffs

Definen cómo una tarea pasa de un agente a otro.

DIFERENTES AGENTES PARTICIPAN EN CADA ETAPA

Planning → Implementation → Validation → Review

VISIÓN GENERAL

Arquitectura conceptual de un sistema multiagente







Proponen

Agentes especializados trabajan en branches aisladas y abren Pull Requests.



Validan y registran

GitHub Actions genera checks, artifacts y logs.



Deciden

CODEOWNERS, reviews y políticas controlan el merge o el despliegue.

CONCEPTOS CLAVE

Seis principios de un sistema multiagente confiable

1. Responsabilidad

Cada agente tiene una función y un alcance claramente definidos.

2. Orquestación

GitHub Actions coordina cuándo y cómo se ejecutan los agentes.

3. Aislamiento

Branches, workflows, permisos y concurrency evitan interferencias.

4. Arbitraje

Checks, CODEOWNERS y reviews permiten resolver conflictos.

5. Observabilidad

Pull Requests, logs y artifacts permiten rastrear cada acción.

6. Recuperación

Retries limitados, rollback y escalamiento humano responden ante fallos.

IDEA PRINCIPAL

Los sistemas multiagente no consisten en ejecutar varios agentes al mismo tiempo.

La confiabilidad depende de cómo GitHub coordina, valida, controla y registra la interacción entre los agentes.

CADA AGENTE

- Tiene una responsabilidad clara
- Trabaja de forma aislada
- Produce evidencia observable
- Entrega sus resultados mediante mecanismos controlados de GitHub